



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026-27, Term-I

Project Problem Statement Submission Form

Section / Team Details:

Section No.: 10

Team No.: 10

Project Title: Concurrent Producer-Consumer System Using Semaphores

Team Members:

Roll Number	Student Name	Signature
2520090085	Vakkalanka Sai Bhaskara Sujit	
2520090087	Duddukuri Bala Charan	
2520090023	Matta Sai Chaitanya	

1. Abstract

In modern multitasking operating systems, coordinating concurrent execution across multiple threads without data corruption is a critical challenge. The Producer-Consumer problem is a classic synchronization paradigm where multiple producer threads generate data into a bounded buffer while consumer threads simultaneously extract it. Without adequate synchronization mechanisms, concurrent access leads to race conditions, buffer overflow, buffer underflow, and deadlocks.

This project implements a robust, multi-threaded Producer-Consumer system in a Linux environment using C and POSIX threads (pthreads). To guarantee thread safety and prevent race conditions, the system utilizes POSIX semaphores (sem_t) and mutex locks. Specifically, counting semaphores track the counts of empty and filled slots in the shared circular buffer, blocking producers when the buffer is full and blocking consumers when it is empty. A binary semaphore enforces mutual exclusion during critical section access, ensuring only one thread modifies the buffer at a time. The expected outcome is a deadlock-free, high-performance concurrent application on Linux that demonstrates seamless synchronization, optimal CPU utilization, and verified data integrity.

2. Problem Statement

1. Concurrent access by multiple threads to a shared bounded buffer leads to race conditions, buffer overflows, and underflows.
2. Operating systems lack inherent coordination to synchronize independent producer and consumer execution.
3. Unsynchronized multi-threading causes deadlocks, inconsistent memory states, and inefficient CPU busy-waiting.
4. This project solves these issues by implementing POSIX semaphores and mutex locks for thread-safe synchronization in Linux.

3. Objectives

1. Design and implement a thread-safe circular bounded buffer in C for shared data storage.
2. Utilize POSIX counting semaphores (empty and full) to coordinate producer and consumer execution without busy-waiting.
3. Implement POSIX mutexes/binary semaphores to enforce mutual exclusion and prevent race conditions within the critical section.
4. Test, evaluate, and benchmark system throughput, deadlock avoidance, and thread synchronization under varying thread ratios in Linux.

4. Proposed Methodology

- **Buffer Initialization:** Allocate a fixed-size shared circular array with head and tail tracking pointers, alongside initializing POSIX semaphores (empty initialized to buffer capacity N, full initialized to 0) and a mutex lock.
- **Producer Workflow:** A producer thread generates an item, performs `sem_wait(&empty)` to ensure available space, acquires the mutex lock to enter the critical section, writes the item into the buffer, updates the write pointer, unlocks the mutex, and signals consumers via `sem_post(&full)`.
- **Consumer Workflow:** A consumer thread performs `sem_wait(&full)` to ensure an item exists, acquires the mutex lock to enter the critical section, reads and removes the item, updates the read pointer, unlocks the mutex, signals producers via `sem_post(&empty)`, and processes the item.
- **Execution & Cleanup:** Spawn multiple worker threads using `pthread_create()`, run concurrent workloads, synchronize completion using `pthread_join()`, and release semaphore/mutex resources.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
<code>pthread_create</code> / <code>pthread_join</code>	Spawns, runs, and synchronizes producer and consumer worker threads.
POSIX Semaphores (<code>sem_wait</code> , <code>sem_post</code> , <code>sem_init</code>)	Coordinates thread execution and manages buffer capacity without busy-waiting.
POSIX Mutex (<code>pthread_mutex_lock</code> , <code>unlock</code>)	Enforces mutual exclusion to protect shared circular buffer memory.
Circular Bounded Buffer	Serves as the fixed-capacity shared memory data structure.

Race Condition & Deadlock Avoidance	Ensures thread safety and strict resource-locking hierarchy.
System Calls (sleep, usleep, printf)	Simulates workload latencies and formats thread execution logs.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520090085	Vakkalanka Sai Bhaskara Sujit	Core circular buffer implementation, memory allocation, and data structures design.
2520090023	Matta Sai Chaitanya	Thread creation, lifecycle management using POSIX pthreads, and thread worker routines.
2520090087	Duddukuri Bala Charan	Semaphore logic implementation (sem_init, sem_wait, sem_post) and mutex locking.
2520090085	Vakkalanka Sai Bhaskara Sujit	Concurrency testing, edge-case validation (full/empty states), and documentation.
2520090087	Duddukuri Bala Charan	Performance profiling, throughput benchmarking, and validation under high workloads.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Primary development, multi-threading execution, and testing environment.
C / C++	System-level programming languages using the GCC compiler (-lpthread).
POSIX Threads Library (pthreads)	Native Linux thread creation and synchronization API.
GDB & Valgrind (Helgrind)	Debugging and verifying race conditions, locks, and thread safety.
Git & GitHub	Version control and collaborative source code management.

8. Expected Outcome

- A functional, compiled Linux CLI application demonstrating concurrent data production and consumption.
- Absolute elimination of race conditions, data corruption, and buffer boundary violations during concurrent execution.
- Removal of CPU-intensive busy-waiting by utilizing native OS-level blocking and wake-up signaling.
- Synchronized and deterministic logging illustrating correct state transitions during empty and full buffer conditions.

9. GitHub Repository

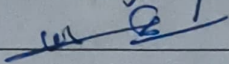
GitHub Repository Link: https://github.com/mattasaichaitanya-gif/KLH-CS-IT_2026-27_Batch10_LinuxProject

Repository Created: Yes

☒ Yes ☐ No

Faculty Name and Signature

Faculty Name: M. Ragheepathi

Signature: 

Remarks: _____

Faculty Use

Project Approved: Approved

☒ Yes ☐ No

Date: 2/9/2024

Faculty Signature: 